COBCheck: Article-Mapped GDPR Compliance Analysis for COBOL

Mainframe COBOL applications are still holding the regulated personal data of banking and insurance systems and were built decades before the enforcement of GDPR and concepts such as consent management or field-level auditability existed. GDPR enforcement against this gap is not hypothetical as the cumulative penalties have reached the order of billions of euro. Also, recovering cross-program data flow in COBOL applications is no longer the open problem it once was - shipped tooling can already trace where personal data moves across programs, copybooks and files. What stands between a recovered flow and an answer to that exposure is a step no current tool performs i.e., deciding which article-level GDPR obligation, if any, an identified flow violates. Current works on source code level compliance analysis tries to target primarily Android and Java, where sources and sinks are known API calls with established privacy semantics. Where as COBOL has no such surface, so both PII source/sink identification and the legal mapping step must be derived from data declarations and usage rather than looked up.

We present COBCheck, a static analysis pipeline that recovers cross-program PII flows in COBOL over a Code-Property-Graph (CPG) and maps each recovered flow to a GDPR article-level obligations, producing article-attributed findings with source evidence. <Need to write the details about the evaluation of the tool, LLM only arm, and combination of LLM and flow recovery arm, and report the results for each of these things>

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Additional Key Words and Phrases: Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
. 2018. COBCheck: Article-Mapped GDPR Compliance Analysis for COBOL. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Banks and insurance companies still run a lot of their core systems on COBOL, and these systems hold a lot of personal data in the form of names, account numbers, health records, sometimes biometric data. GDPR puts legal obligations on how this kind of data can be processed, stored and moved. The problem is that mainframe systems were never built with GDPR in mind. Practitioner reports on mainframe modernization say clearly that these systems have no built-in way to handle consent management, data-subject access requests, or field-level auditing [4]. The relationships between data fields are not written down anywhere as a clean schema. They live inside the COBOL programs themselves, inside copybooks, and inside VSAM files. So if someone wants to know whether a particular flow of data breaks Article 6 or Article 32 of GDPR, the only place to actually check is the source code. This is not a small compliance question either — GDPR fines have already

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

added up to billions of euro since the regulation started being enforced [25], so getting this answer wrong is not just an engineering mistake, it has legal consequences too.

There is already research on static analysis for privacy compliance, but almost all of it is built for a different kind of language. GDPR-tailored taint analysis, for example, works by taking a policy that the data controller has already written down, tracking where data leaks to using taint analysis, and then checking the leak against that policy [11, 13]. This approach works well because in Android or Java, sources and sinks are API calls, and those API calls already carry a known privacy meaning and sometimes like *getLocation()* is obviously a source, and *Log.d()* is obviously a sink. This is also why taint analysis tools built for Android, like the well-known ones surveyed in recent GDPR benchmarking work, can reach high precision on this kind of code [3]. However it is not possible to do similar analysis in COBOL. A **MOVE** statement or a **WRITE** to a file does not tell you anything about privacy by itself. Before any taint analysis can even start, the tool first has to decide which data elements actually count as personal data, and it has to make that decision from PICTURE clauses, field names, and copybook layout, not from an API signature. And even inside the settings where taint analysis already works well, the literature itself says that plain taint analysis can only tell you that a leak exists - it cannot tell you how the data was handled or whether it was protected, which is exactly the thing you actually need for a compliance decision [2]. To the best of our knowledge for COBOL, nobody has built this extra step yet, in either direction - not the personal-data identification step, and not the compliance-decision step that comes after.

Some of this gap has already started closing, but only the easier half of it. Recovering how data moves between COBOL programs - across **CALL** statements, shared copybooks, shared files - is now something shipped tooling can already do. This has been demonstrated on the AWS CardDemo application, which is reported to contain 59 separate COBOL data structures spread across its copybooks, and which we are also using as one of our evaluation applications [1]. Commercial static analysis tools for COBOL, separately, already maintain a full metadata repository over control flow, data flow, and file dependencies across an entire mainframe codebase, and can do impact analysis and code slicing on top of that [15]. What none of these tools do is take one more step and say which legal obligation a recovered data dependency actually falls under. Knowing that a customer's SSN moves from *PROGA* into a shared file, and that *PROGB* later reads that same file, is a fact about how data moves through the system. It is a different fact, and a harder one to get right, to say that this particular movement is a violation of Article 32's requirement around security of processing, rather than Article 6's requirement around a lawful basis for processing [9], or that it is not a violation of anything at all. This second decision is the one that actually decides whether a recovered flow is useful for compliance purposes, and it is the one nobody has built yet for COBOL.

This is the central claim of the paper, and it has three parts. First, an LLM reading raw COBOL source one file at a time cannot see a flow that crosses a **CALL** boundary, a shared copybook, or a shared file, since it never sees the two programs connected - this is a visibility problem, not a legal-reasoning one, and it is the failure we expect from the LLM only arm [3]. Second, this is exactly what the code property graph is built to fix: once **CALL**, **FILE_IO**, and **COPYBOOK** edges connect the codebase into one graph, a cross-program flow is just as trace as an intra-program one, so flow recovery itself should not degrade cross-program. The third claim is mapping a data flow detection to a GDPR obligation might not be better performing when we are having violation across programs. We try to minimize this performance gap by comparing entire function bodies/COBOL programs for complete detection of cross program GDPR violations.

We test this with COBCheck. It identifies personal data from COBOL data declarations, since there is no API surface to read this off, unlike Android or Java. It builds a code property graph and recovers intra- and cross-program flows over it using **CALL**, **FILE_IO**, and **COPYBOOK** edges. A separate article-mapping layer then checks every recovered flow — and, for the obligation

above, every **CALL** boundary directly, without needing a flow at all — against a registry of GDPR obligations grounded in the article text [9] and organized along principle groupings from an existing GDPR conceptual model [26]. Each obligation carries its own code-level condition and its own evidence trail, kept apart from the taint engine's evidence. This separation is what lets one pipeline support three evaluation arms — an LLM with no recovered flow facts, flow-recovery with no mapping, and the full pipeline — scored separately rather than pooled, following the evaluation discipline argued for in prior static analysis work on personal data [14].

This paper's contributions:

- A source-level GDPR checker for COBOL that identifies personal data from declarations, not an API surface, unlike existing GDPR-tailored taint analysis [11, 13].
- An article-mapping layer grounded in GDPR's text [9] and an existing conceptual model [26], kept separate from flow recovery so mapping and detection errors can be measured apart.
- A structural split between obligations that are cross-program only by trigger condition and those that are cross-program by evidentiary necessity, giving the evaluation a falsifiable target instead of a pooled score.
- An evaluation plan on an injected-violation benchmark over CardDemo and GenApp [1], comparing LLM-only, flow-recovery-only, and the full pipeline, reported per obligation as in prior Android GDPR benchmarks [3]. *(Results pending.)*

2 Research Questions

2.1 RQ1 Detection Quality:

How accurately does COBCheck detect GDPR-relevant PII flows in COBOL source? We measure this per obligation as precision, recall and F1 scores against an injected-violation ground truth, reported per article rather than pooled. This is because pooling would obscure which obligations the pipeline handles well and what are the detections that pipeline does not handle.

2.2 RQ2 Locus of Error

When a compliance decision is wrong, does the error originate in flow recovery - failing to trace where personal data travels - or in obligation mapping - tracing the flow correctly but attributing the wrong article or none? We try to answer this by scoring article attribution accuracy separately from flow detection, so a mapping failure is visible as such rather than folded into an aggregated score. This RD is the paper's central claim: mapping is essential, not recovery, is where correctness in source-level analysis is won or lost.

2.3 RQ3 Evidence locality

Does detection accuracy differ between obligations whose evidence is local to a single program or the obligation is spread across multiple program boundaries? We compare cross-program against intra-program recall across three arms - an LLM applied to source without recovered flow facts. flow recovery without article mapping and the full pipeline. This comparison will be used for testing the capability of cross-program flow recovery to narrow the gap that source-level LLM reasoning alone cannot.

2.4 RQ4 Practical viability

Is COBCheck's output reliable enough to be usable in an audit setting? We measure this along two axes - the false positive rate on unmodified, uninjected copies of the chosen evaluation applications and inter-rater agreement between two human raters on the injected-violation ground truth scored against a threshold fixed before annotation begins. If the tool over-reports on clean

code or if the ground truth cannot be reproduced independently, it is not usable regardless of its recall value.

3 Related Work

3.1 GDPR-Aware Static Analysis

Prior work has explored the use of static analysis for reasoning about GDPR compliance. Taint analysis can be used to identify potential personal-data leakages and subsequently determine whether the corresponding flows satisfy specified privacy policies [13], [11]. Source code annotations can be used to support static analysis of personal-data processing [14], while reconstruction of tainted-data flows to provide more precise evidence for privacy analysis [12]. These approaches establish the importance of connecting program-level data flows with data-protection requirements, but primarily limited target languages and environments where privacy-relevant sources, sinks, policies, or annotations can be explicitly identified. COBCheck instead targets COBOL, where such information must be derived from data declarations, copybooks, file operations, and inter-program interactions, and further separates flow recovery from article-level GDPR attribution.

3.2 Privacy Taint Analysis and Cross-Component Data Flow

Taint-analysis systems have also addressed privacy-sensitive flows that cross program or component boundaries. FlowDroid [5] provides context, flow, field, object, and lifecycle-sensitive taint analysis for Android, while IccTA [18] and Amandroid [27] extend data-flow reasoning across Android components. These systems demonstrate the importance of analyzing data beyond individual procedures or components. However, Android provides well-defined APIs and component semantics that expose many privacy-relevant sources and sinks. But on the other hand COBOL is not having similar privacy-oriented API surface, with personal data instead represented through declared data items, copybooks, program calls and files. COBCheck therefore uses cross-program flow recovery as evidence for a subsequent for identifying which specific GDPR obligation is engaged by the recovered flow.

3.3 Static Analysis of COBOL and Mainframe Applications

COBOL and mainframe analysis has a mature ecosystem for recovering program dependencies and data relationships. Tools such as Micro Focus Enterprise Analyzer provide analysis of COBOL programs, including program relationships and data-flow dependencies [20], while AWS Transform provides data-analysis capabilities that recover lineage across COBOL programs, JCL jobs, datasets, and data structures [1]. Recent academic work further demonstrates continued interest in program analysis for enterprise COBOL, including the use of program analysis to augment LLM-based COBOL-to-Java translation [7] and path-based COBOL program analysis through an agentic tool [22]. These works establish that recovering program structure and cross-program data relationships is an active research and engineering capability. COBCheck builds on this capability but addresses a different downstream problem—determining whether a recovered personal-data flow corresponds to a specific GDPR obligation and providing source-level evidence for that attribution.

3.4 Automated GDPR Compliance Detection

Recent work has explored automated approaches for connecting source-code analysis with GDPR compliance. Khedkar and Bodden investigate static analysis for data protection in Android applications, using taint analysis to identify personal-data sources and reason about their subsequent flows and processing [16]. More recently, Khedkar et al. study the challenges of automating privacy assessments and use static analysis to extract privacy-relevant information from Android

applications, highlighting the gap between technical source-code evidence and the legal expertise required for privacy assessment [17]. GDPR-Bench-Android further introduces a benchmark for evaluating automated GDPR-compliance detection at the level of individual GDPR articles, considering static-analysis, LLM-based, retrieval-augmented, and agentic approaches [3]. These works provide important foundations for automated privacy and compliance analysis, but primarily consider Android applications with platform-specific APIs and structures. COBCheck extends this line of work to COBOL, where privacy-relevant information must be recovered from data declarations, copybooks, program interactions, and files, and explicitly separates the recovery of technical evidence from its mapping to GDPR obligations.

3.5 Regulatory requirements and legal traceability

Though not a direct relation with our, current exploration direction, regulatory requirements engineering has studied how legal provisions can be translated into software requirements and subsequently connected to technical artifacts. Negri-Ribalta et al. [21] provide a systematic mapping of research on requirements engineering for GDPR compliance, highlighting the challenges of transforming legal provisions into precise requirements suitable for software engineering activities given the ambiguity of regulatory requirements. Etezadi et al. [8] further investigate automated traceability between software requirements and legal provisions, including GDPR, using machine-learning and LLM-based approaches. COBCheck differs in the artifact being traced - rather than linking natural-language software requirements to legal provisions, it investigates whether evidence recovered directly from source-code data flows can be attributed to specific regulatory obligations.

3.6 Positioning

Existing research therefore provides complementary capabilities for GDPR-aware static analysis, privacy-sensitive data-flow recovery, COBOL program analysis, automated compliance detection, and legal-requirement traceability. COBCheck combines these perspectives in a setting not addressed by the existing approaches - article-level GDPR compliance analysis over COBOL source in the presence of cross-program data flows. In particular, COBCheck treats flow recovery and article attribution as distinct analysis stages, enabling the evaluation to determine whether an incorrect compliance decision originates from missing flow evidence or from incorrect attribution of that evidence to a regulatory obligation.

4 Methodology

COBCheck is implemented as a static analysis pipeline that combines COBOL program parsing, program-graph construction, PII identification, field-sensitive taint analysis, and GDPR-oriented rule checking. The methodology follows established principles from static program analysis while adapting them to the structure and semantics of COBOL programs. In particular, the use of a unified graph representation is motivated by Code Property Graphs (CPGs), which combine abstract syntax, control-flow, and program-dependence information to support security-oriented graph analysis [28]. Similarly, source-to-sink taint propagation and field-sensitive tracking are well-established techniques for identifying potentially unsafe flows of sensitive information in static analysis [6].

The resulting methodology is therefore not intended as a direct reimplementation of a single prior system. Instead, COBCheck combines established static-analysis abstractions with COBOL-specific program representations and GDPR-specific rules. The overall workflow consists of six main stages: (i) COBOL parsing and program representation, (ii) copybook resolution, (iii) construction of a Code Property Graph (CPG), (iv) PII identification, (v) field-sensitive taint propagation, and (vi) GDPR

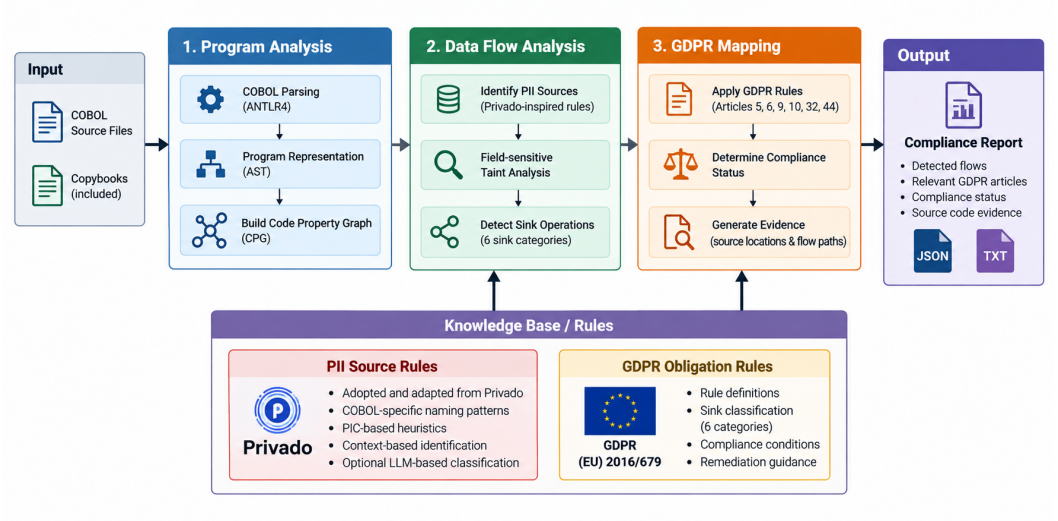


Fig. 1. COBCheck Architecture

rule evaluation and violation reporting. The diagrammatic representation for the architectural flow of COBCheck is in figure 1.

4.1 System Overview

Given one or more COBOL source directories, COBCheck first discovers COBOL source files and resolves referenced copybooks. Each program is then parsed into a typed intermediate representation. The resulting representation is used to construct a graph containing data, procedural, control-flow, data-dependence, inter-program, and file-related relationships.

PII identification is subsequently performed over data-item nodes in the graph. Identified PII fields are used as taint sources, while operations such as database writes, file writes, logging, external transfers, and messaging operations are treated as potential sinks. A field-sensitive taint engine propagates PII through selected data-flow relationships and records source-to-sink paths. Finally, the detected flows are evaluated against a set of GDPR-oriented rules derived from requirements that can be operationalized as observable static program conditions.

This design follows the general source–flow–sink structure commonly used in static information-flow and taint analyses [6], while adapting the source and sink definitions to COBOL constructs and the data-processing operations relevant to GDPR analysis.

4.2 COBOL Parsing and Program Representation

COBCheck uses an ANTLR4-based COBOL parser to transform source programs into a structured abstract syntax representation. The parser preserves source-level information such as the program name, source file, line, and column for the represented elements. This source information is later used to associate analysis results with their corresponding locations in the original COBOL program.

Data declarations are represented explicitly as data-item nodes. Each data item records attributes including its level number, name, PIC description, section, copybook association, and hierarchical children. The implementation distinguishes data items occurring in sections such as `FILE`, `WORKING-STORAGE`, `LINKAGE`, and `LOCAL-STORAGE`.

Procedure-level constructs are represented explicitly for operations relevant to data-flow analysis. These include `MOVE`, `COMPUTE`, `CALL`, `READ`, `WRITE`, `DISPLAY`, conditional constructs, and `PERFORM` statements. The representation also preserves external operations such as `EXEC SQL`, `EXEC SQL IMS`, and `EXEC CICS`. Statements that are not represented by a specialised node are retained as generic statements together with their original source text.

This explicit representation is important for legacy-language analysis because the subsequent flow analysis operates on language constructs rather than on raw source-text matching alone. It also provides a stable intermediate representation through which source locations, data declarations, and procedure operations can be related.

Copybooks are resolved before graph construction. The resolver searches the configured copybook and source directories and associates resolved copybook content with the corresponding program representation. This allows shared COBOL data structures to participate in the subsequent graph construction and data-flow analysis.

4.3 Code Property Graph Construction

The parsed representation is transformed into a Code Property Graph (CPG) implemented using a NetworkX directed multigraph. The choice of a CPG is motivated by prior work showing that combining abstract syntax, control-flow, and program-dependence representations into a unified graph enables security properties to be expressed as graph-based analyses [28].

The graph contains nodes for data items and relevant program operations, including `MOVE`, `COMPUTE`, `CALL`, `READ`, `WRITE`, `DISPLAY`, SQL and IMS operations, CICS operations, conditionals, and procedure boundaries. It also contains explicit nodes for call sites and file read/write operations.

Edges are labelled according to their semantic relationship. The implementation includes `AST`, `CFG`, `PDG`, `CALL`, `RETURN`, `COPYBOOK`, `FILE_IO`, and containment relationships. AST and CFG edges represent syntactic and control-flow structure, whereas PDG and the inter-program relationships represent data movement relevant to taint propagation.

The graph builder constructs the graph in multiple passes. Initially, program boundaries, data items, and statement nodes are created. Control-flow and program-dependence relationships are then added. Finally, relationships crossing program boundaries are constructed, including call relationships, copybook relationships, and relationships between programs that access the same logical files.

The graph also records attributes used by the later analyses. In particular, nodes can be marked as PII sources, potential sinks, sanitizers, guards, or retention-related markers. Sink classification is based on the semantics represented by the operation. For example, `DISPLAY` operations are treated as logging-like sinks, database modification statements are treated as database sinks, and selected `CALL`, CICS, and file operations can represent messaging, network-transfer, or external-service sinks.

The CPG therefore serves as the common representation shared by the PII, taint, and GDPR analyses. This follows the motivation of CPG-based static analysis, where multiple program properties can be inspected through a common graph representation rather than through independent analyses [28].

4.4 PII Identification

PII identification is performed before taint propagation so that personal-data fields can be used as taint sources. COBCheck adopts the source-oriented organization used by privacy-focused static analysis tools such as Privado, where personal-data elements are classified into semantic categories and identified using source rules and patterns [24]. Privado similarly models privacy analysis as a

source-to-sink data-flow problem in which identified personal-data sources are tracked toward destinations such as databases, APIs, logging functions, messaging queues, and third parties [23].

Rather than directly applying programming-language-specific Privado rules, COBCheck adapts this source-oriented approach to COBOL. The implementation defines COBOL-specific semantic patterns for the corresponding personal-data categories and combines them with information available in COBOL data declarations, including field names and PIC clauses. This adaptation is necessary because the syntax, naming conventions, and data representation mechanisms of COBOL differ substantially from those targeted by contemporary source-analysis rules.

First, the implementation applies Privado-inspired source categories and COBOL-adapted semantic rules to identify candidate personal-data fields. The identifier processing normalizes COBOL field names and matches them against category-specific patterns. The resulting categories cover common personal identifiers, contact information, financial information, health-related information, location information, and other personal-data classes represented by the adopted taxonomy.

Second, COBOL-specific name heuristics are applied to identify additional personal-data fields that may not be captured by the generalized source patterns. These heuristics include identifiers for national identification numbers, financial identifiers, contact information, dates of birth, and other categories relevant to GDPR analysis.

Third, PIC clauses are considered as additional evidence. The tagger combines field-name information with COBOL picture characteristics to identify fields whose representation is consistent with particular personal-data categories.

The implementation also performs deterministic contextual analysis over nearby graph nodes. A field can receive category evidence from related identifiers, with a category being accepted only when sufficient contextual evidence is present. An optional Groq-based language-model fallback can additionally classify uncertain fields using the field name, PIC information, and local context.

Only elementary data-item nodes are independently tagged as PII sources. Group items are not treated as independent PII sources; however, the graph contains containment relationships that allow taint to propagate from an elementary field to its containing structures.

4.5 Sink Classification

Privacy-oriented static analyses commonly model destinations of personal data as sinks, including storage systems, logging operations, APIs, messaging infrastructure, and external services [23]. Prior work on GDPR analysis of Android mHealth applications has similarly demonstrated the value of connecting regulatory requirements to concrete data-processing and transmission behaviour in application code [10].

COBCheck adapts this source-to-sink abstraction to the operations available in COBOL and defines six sink categories: `LOGGING`, `FILE_STORAGE`, `DATABASE`, `NETWORK_TRANSFER`, `MESSAGING`, and `CONTENT_OR_SERVICE`. These categories are an operational taxonomy introduced by COBCheck rather than a direct reproduction of a single prior sink taxonomy.

`DISPLAY` operations are classified as `LOGGING`. File `WRITE` operations are classified according to their surrounding file or record identifiers: logging-like names are mapped to `LOGGING`, externally oriented names are mapped to `NETWORK_TRANSFER`, and other file writes are mapped to `FILE_STORAGE`. SQL and IMS database modification operations are classified as `DATABASE` sinks. Literal `CALL` targets containing messaging-related identifiers are classified as `MESSAGING`, while targets associated with APIs, sockets, HTTP, or similar service interfaces are classified as `CONTENT_OR_SERVICE`. Relevant CICS operations are classified according to their communication semantics.

The purpose of this classification is to provide a uniform abstraction over heterogeneous COBOL operations so that GDPR-oriented rules can be defined in terms of the destination of personal data rather than individual source-code statements.

4.6 Field-Sensitive Taint Analysis

The central data-flow analysis in COBCheck is a field-sensitive, forward taint analysis. Each elementary PII field identified by the PII tagger is treated as an independent taint source. The purpose of the analysis is to determine whether information originating from a PII field can reach an operation that represents a security, storage, communication, or other GDPR-relevant sink.

The choice of taint analysis is supported by established static information-flow analysis techniques, in which sensitive sources are tracked through program operations until they reach security-relevant sinks. FlowDroid, for example, demonstrates the use of field-sensitive and flow-sensitive static taint analysis for tracking sensitive information [6]. Inter-component extensions such as IccTA further demonstrate the use of taint propagation across component boundaries for privacy-flow analysis [19]. COBCheck adopts the underlying source–propagation–sink abstraction, but defines its propagation relationships and sources/sinks according to COBOL program structure.

The taint engine uses a worklist-based propagation procedure. Starting from each identified PII source, taint is propagated through graph relationships that represent data movement. Importantly, not every edge in the CPG is considered a taint-carrying edge. The analysis uses `PDG`, `CALL`, `RETURN`, `FILE_IO`, `COPYBOOK`, and `CONTAINS` edges for propagation, while purely structural `AST` and `CFG` edges are not treated as evidence that a value has been transferred.

This separation follows the general distinction between program structure and data dependence used in static program analysis and CPG construction [28]. Consequently, syntactic or control-flow reachability alone does not imply that a PII value has been transferred.

4.7 Intra-Program Propagation

Within a program, taint is propagated through data-dependence relationships created for assignments and data-processing operations. For example, a `MOVE` relationship between two data items can transfer taint from the source field to the destination field. Similar relationships are constructed for relevant `COMPUTE`, `READ`, `WRITE`, `DISPLAY`, and external-operation nodes.

Containment relationships are also considered during propagation. This allows taint associated with an elementary data item to be reflected in the corresponding group structure when that structure participates in a subsequent operation.

The analysis is therefore field-sensitive at the level supported by the COBOL representation: the provenance of individual elementary fields is maintained rather than collapsing all data in a program or record into a single taint state. Field sensitivity is a well-established mechanism for improving the precision of static information-flow analyses [6].

4.8 Inter-Program Propagation

COBCheck extends taint propagation across program boundaries. For `CALL` operations, the graph builder relates caller arguments to the corresponding fields in the callee's `LINKAGE` section. These relationships are represented using `CALL` and `PDG`-related edges. Return relationships are also represented explicitly.

The implementation additionally handles group parameters by matching relevant child fields between caller arguments and callee linkage structures. This allows PII contained inside structured COBOL parameters to be followed across program boundaries rather than treating the group as an opaque value.

Inter-component and inter-procedural propagation is motivated by prior privacy-flow analyses showing that limiting taint tracking to an individual component can miss flows that cross program boundaries [19]. In COBCheck, the analogous boundary is a COBOL program invocation rather than an Android component.

4.9 File-Based Data Flow

COBCheck also models data movement through shared files. File relationships are constructed using the file and record information available from the COBOL declarations. When a write operation and a read operation refer to normalized matching file or record identifiers, a `FILE_IO` relationship can connect the producing operation with the consuming operation.

This enables the taint analysis to follow PII across program boundaries even when the programs do not directly call one another. Thus, a value written by one COBOL program and subsequently read by another can remain part of the same taint flow.

This extension is specific to the storage-oriented communication model of the target language and represents an adaptation of conventional inter-procedural information-flow analysis to a COBOL environment.

4.10 Source-to-Sink Flow Reconstruction

For each detected source-to-sink flow, COBCheck records the originating PII field, PII category, source program, sink operation, sink program, and sink kind. The taint engine reconstructs a representative path between the source and sink using the taint-carrying portion of the graph. NetworkX shortest-path analysis is used for this path reconstruction.

The resulting flow record also captures whether a sanitizer, consent guard, or retention marker occurs along the representative path. Sanitizer and guard information is derived from graph annotations generated during CPG construction. The engine performs flow deduplication so that intermediate PII-labelled nodes do not unnecessarily produce duplicate reports for the same underlying movement of data.

The resulting source-to-sink path is used as evidence for the subsequent GDPR rule evaluation rather than treating the rule engine as an independent pattern-matching component.

4.11 Operationalization of GDPR Requirements

The GDPR analysis is deliberately formulated as an operationalization of selected regulatory requirements rather than as a claim of complete automated legal compliance verification. The General Data Protection Regulation defines principles including lawfulness, purpose limitation, data minimisation, integrity and confidentiality in Article 5, while Articles 6, 9, 10, 32, and 44 address lawful bases, special categories of personal data, criminal-offence data, security of processing, and international transfers, respectively [9].

For each selected obligation, COBCheck identifies a code-level condition that can provide static evidence relevant to that obligation. This design allows the analysis to remain grounded in observable program behaviour while avoiding unsupported claims that a source-code pattern alone proves a legal violation.

For example, a PII flow reaching a logging or display operation without a detected sanitization marker can generate a finding associated with Article 5(1)(f). A corresponding Article 32 finding is reported separately for the same technical condition. Similarly, PII reaching a file or database sink without a detected sanitization marker is checked against the corresponding at-rest protection conditions.

The lawful-basis check is deliberately treated as an absence-of-evidence check. The analyzer reports the absence of a detectable consent or authorization indicator rather than concluding that

the processing is legally unlawful. This distinction is important because the existence of a lawful basis cannot in general be established solely from a source-code pattern.

Similarly, the retention check reports the absence of a recognizable retention or deletion marker rather than claiming that the underlying retention period necessarily violates the GDPR. The purpose-limitation and international-transfer checks are likewise expressed as indicators requiring interpretation of the surrounding application context.

Special-category PII is checked separately for Article 9-oriented analysis, while fields classified as criminal-offence data are identified for Article 10-oriented review. External-looking sinks can generate findings associated with international transfer requirements, but the static analysis does not attempt to establish whether a specific transfer mechanism or derogation is legally available.

COBCheck also performs structural analysis of cross-program data movement. A flow crossing a program boundary can be examined for potential purpose changes, while the call-boundary analysis checks whether PII passed through a `CALL` interface is subsequently used by the receiving program. These checks provide static indicators for further review rather than complete legal determinations.

Thus, the GDPR rule layer should be understood as a mapping from regulatory requirements to statically observable program conditions. The regulatory text provides the normative basis for the selected requirements, while the implementation defines conservative, code-level indicators that can be evaluated automatically.

4.12 Operationalization of GDPR Requirements

The GDPR analysis is deliberately formulated as an operationalization of selected regulatory requirements rather than as a claim of complete automated legal compliance verification. The General Data Protection Regulation defines principles including lawfulness, purpose limitation, data minimisation, storage limitation, and integrity and confidentiality in Article 5, while Articles 6, 9, 10, 32, and 44 address lawful bases, special categories of personal data, criminal-offence data, security of processing, and international transfers, respectively [9]. Prior work has demonstrated that selected GDPR requirements can be related to observable application-level behaviour and analysed using static program and data-flow analysis [10**?**].

COBCheck follows this general approach while adapting it to COBOL program structure and data-processing operations. Rather than attempting to determine legal compliance directly, each selected GDPR requirement is translated into a code-level condition that can be evaluated using the CPG, PII classifications, and source-to-sink taint flows. The resulting mapping is implemented as executable rules in the GDPR rule engine. Table 1 summarizes the implemented mapping between the selected GDPR articles, their corresponding static conditions, and the evidence that can clear or qualify a finding.

The mapping is evaluated over the evidence produced by the preceding analysis stages. In particular, R1–R8 operate on detected PII flows and their associated source, sink, category, and representative path, while R9 additionally examines the structure of a `CALL ... USING` boundary and the callee body. Thus, the rule engine does not independently rediscover data flows; it evaluates previously established analysis evidence against rule-specific conditions.

For example, a PII flow reaching a logging or display operation without a detected sanitization marker can generate both an R1 finding under Article 5(1)(f) and an independent R1B finding under Article 32. The same source-to-sink analysis is therefore capable of providing evidence for multiple related regulatory obligations without requiring separate data-flow analyses.

The lawful-basis check is deliberately treated as an absence-of-evidence check. R2 reports the absence of a detectable consent or permission guard rather than concluding that the processing

Table 1. Operationalization of selected GDPR requirements in COBCheck.

Rule	GDPR Article	Static detection condition	Condition that clears / qualifies the finding
R1	Art. 5(1)(f)	Tainted PII reaches a logging/display sink (`DISPLAY` or a log/audit-named `WRITE`).	Recognised masking or encryption operation on the taint path.
R1B	Art. 32	Same technical condition as R1, reported as an independent security-of-processing finding.	Same as R1.
R2	Art. 6(1)	Tainted PII reaches a sink with no detectable consent/permission guard on the path.	A detectable lawful-basis guard on the path.
R3	Art. 44	Tainted PII reaches an external-looking sink, including network transfer, messaging, or external content/service.	No automatic clearance; requires human confirmation of the applicable transfer mechanism.
R4	Art. 5(1)(f)	Tainted PII reaches a file or database sink.	Masking or encryption operation on the taint path.
R4B	Art. 32	Same technical condition as R4, reported as an independent security-of-processing finding.	Same as R4.
R5	Art. 9	PII classified as an Article 9 special category reaches a sink without an explicit-consent guard.	Explicit-consent guard; masking alone does not clear the finding.
R6	Art. 10	Criminal-offence-related PII reaches a sink.	No automatic clearance; flagged for manual legal review.
R7	Art. 5(1)(e)	Tainted PII reaches an at-rest sink without a retention, expiry, or purge marker on the path.	A retention marker on the path.
R8	Art. 6(4)	Source and sink programs differ and the sink indicates a potential purpose shift (external, messaging, or database).	No automatic clearance; flagged for review.
R9	Art. 5(1)(c)	PII crosses a `CALL ... USING` boundary into a callee and is never referenced in the callee.	The callee actually uses the passed field.

is legally unlawful. Similarly, R7 reports the absence of a recognisable retention, expiry, or purge marker rather than establishing that the underlying retention period necessarily violates the GDPR.

The external-transfer and criminal-offence checks are likewise conservative. R3 identifies an external-looking destination but cannot determine from source code alone whether an appropriate transfer mechanism under Articles 45–49 exists. R6 identifies criminal-offence-related data reaching a sink but does not attempt to establish whether the processing is supported by the required official authority or legal basis. These findings are therefore explicitly reported for further review.

R5 is treated separately because the presence of Article 9 special category data is itself relevant to the applicable processing conditions. Consequently, masking or encryption does not by itself clear an R5 finding; the implemented check instead looks for an explicit-consent guard.

Two rules require cross-program evidence for structurally different reasons. R8 is cross-program by its trigger condition: it requires the source and sink programs to differ before a potential purpose shift is considered. R9 is cross-program by evidentiary necessity: determining whether a PII field passed through a `CALL ... USING` interface is subsequently unused requires information from both the caller and the callee. Thus, R9 cannot be reduced to a conventional single-program source-to-sink flow check.

Finally, each generated violation records its locality as either intra-program or cross-program, based on the resolved source and sink programs. This allows the same rule framework to retain the distinction between local and inter-program evidence during subsequent evaluation.

4.13 Violation Reporting

For each reported finding, COBCheck records structured information including the violation identifier, rule identifier, relevant GDPR article, severity, PII field and category, source program and location, sink type and location, representative flow path, sanitization state, consent and retention indicators, rationale, remediation guidance, locality information, and detected boundary crossings.

The final results are emitted in JSON format in addition to a human-readable command-line summary. This representation preserves the source-to-sink evidence required to inspect a finding and trace it back to the corresponding COBOL statements.

The resulting methodology therefore connects each reported finding to three forms of evidence: (i) an established static-analysis abstraction such as graph-based program representation or taint propagation, (ii) a concrete source-to-sink flow extracted from the COBOL program, and (iii) a GDPR requirement whose relevant condition has been explicitly operationalized.

The implementation does not claim complete GDPR coverage or formal verification of legal compliance. Obligations that cannot be reliably operationalized from static COBOL evidence are outside the scope of the implemented checks.

5 Evaluation

6 Limitations

7 Conclusion

References

[1] Amazon Web Services. 2026. Data Analysis with AWS Transform for Mainframe: Understand Your Application's Data Landscape. AWS re:Post. https://repost.aws/

[2] Anonymous. 2024. Toward an Android Static Analysis Approach for Data Protection. arXiv:2402.07889. https://arxiv.org/abs/2402.07889

[3] Anonymous. 2025. GDPR-Bench-Android: A Benchmark for Evaluating Automated GDPR Compliance Detection in Android. arXiv:2511.00619. https://arxiv.org/abs/2511.00619

[4] Archon. 2026. COBOL Modernization and Migration: A Complete Enterprise Guide. Company blog. Practitioner source; to be replaced or supplemented with a citable source before submission..

[5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 259–269. doi:10.1145/2594291.2594299

[6] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 259–269. doi:10.1145/2594291.2594299

[7] Venkatesan T. Chakravarthy, Anamitra Roy Choudhury, Dinesh Garg, Vini Kanvar, Shivmaran S, Aditya Raghuvanshi, Yogish Sabharwal, and Amith Singhee. 2026. Enterprise-Scale COBOL-to-Java Translation: LLMs Augmented with Program Analysis. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 775–785. doi:10.1145/3786583.3786915

[8] Romina Etezadi, Sallam Abualhaija, Chetan Arora, and Lionel Briand. 2026. Classifier or prompt: A case study on legal requirements traceability. *Empirical Software Engineering* 31, 4 (2026), 85. doi:10.1007/s10664-026-10827-1

[9] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L 119. https://gdpr-info.eu/

[10] Ming Fan, Le Yu, Sen Chen, Hao Zhou, Xiapu Luo, Shuyue Li, Yang Liu, Jun Liu, and Ting Liu. 2020. An Empirical Evaluation of GDPR Compliance Violations in Android mHealth Apps. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 253–264. doi:10.1109/ISSRE5003.2020.00032

[11] Pietro Ferrara, Luca Olivieri, and Fausto Spoto. 2018. Tailoring Taint Analysis to GDPR. In *Privacy Technologies and Policy (APF 2018) (Lecture Notes in Computer Science)*. Springer. doi:10.1007/978-3-030-02547-2_5

[12] Pietro Ferrara, Luca Olivieri, and Fausto Spoto. 2021. Static Privacy Analysis by Flow Reconstruction of Tainted Data. *International Journal of Software Engineering and Knowledge Engineering* 31, 7 (2021), 973–1016. doi:10.1142/S0218194021500303

[13] Pietro Ferrara and Fausto Spoto. 2018. Static Analysis for GDPR Compliance. In *Proceedings of the Italian Conference on Cybersecurity (ITASEC)*. Milan, Italy.

[14] Kalle Hjerppe, Jukka Ruohonen, and Ville Leppänen. 2020. Annotation-Based Static Analysis for Personal Data Protection. In *Proceedings of the IFIP Summer School on Privacy and Identity Management (IFIP Advances in Information and Communication Technology)*. Springer. doi:10.1007/978-3-030-42504-3_10

[15] IN-COM Data Systems. 2025. Top COBOL Static Code Analysis Solutions for Mission-Critical Systems. Company blog. Practitioner source; used for the commercial-baseline argument (Micro Focus Enterprise Analyzer)..

[16] Mugdha Khedkar and Eric Bodden. 2024. Toward an Android Static Analysis Approach for Data Protection. *arXiv preprint arXiv:2402.07889* (2024). doi:10.48550/arXiv.2402.07889

[17] Mugdha Khedkar, Michael Schlichtig, Nihad Atakishiyev, and Eric Bodden. 2026. Between Law and Code: Challenges and Opportunities for Automating Privacy Assessments. *Automated Software Engineering* 33, 2 (2026), 56. doi:10.1007/s10515-026-00601-4

[18] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Octeau, and Patrick McDaniel. 2015. IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 280–291. doi:10.1109/ICSE.2015.48

[19] Li Li, Alexandre Bartel, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Octeau, and Patrick McDaniel. 2014. I Know What Leaked in Your Pocket: Uncovering Privacy Leaks on Android Apps with Static Taint Analysis. In *Proceedings of the 2014 International Conference on Software Engineering*.

[20] Micro Focus. 2024. Enterprise Analyzer. https://www.microfocus.com/. Application analysis and data-flow analysis tooling for COBOL applications.

[21] Claudia Negri-Ribalta, Marius Lombard-Platet, and Camille Salinesi. 2024. Understanding the GDPR from a requirements engineering perspective—a systematic mapping study on regulatory data protection requirements. *Requirements Engineering* 29 (2024), 523–549. doi:10.1007/s00766-024-00423-4

[22] Sameer Pimparkhede, Pankaj Kumar Kalita, Diptikalyan Saha, Toshiaki Yasue, and Fumiko Satoh. 2026. Path-Guider: An IDE-based Agentic AI Tool for Path-based COBOL Program Analysis. In *Proceedings of the 2026 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Tool Demonstrations Track.

[23] Privado AI. 2024. Data Flow Discovery. https://help.privado.ai/en/articles/7202101-data-flow-discovery. Privado documentation.

[24] Privado AI. 2024. Sources. https://docs.privado.ai/understanding-rules/sources. Privado documentation.

[25] Secure Privacy. 2026. GDPR Compliance in 2026: The Complete Guide. Company blog. Practitioner source; to be replaced or supplemented with a citable enforcement-tracker figure before submission.. https://secureprivacy.ai/

[26] Damiano Torre, Mauricio Alferez, Ghanem Soltana, Mehrdad Sabetzadeh, and Lionel Briand. 2021. Modeling Data Protection and Privacy: Application and Experience with GDPR. *Software and Systems Modeling* 20, 6 (2021), 2071–2087. doi:10.1007/s10270-020-00819-1

[27] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Transactions on Privacy and Security* 21, 3 (2018), 1–32. doi:10.1145/3183575

[28] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy*. 590–604. doi:10.1109/SP.2014.44

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009